

The report of Writeup project

Yang Li

1/9/2026

Introduction

The objective is to develop a predictive model to forecast the manner in which an exercise was performed, identified by the “class” variable in the training dataset. The report should detail:

1. The methodology used to build the model.
2. The application of cross-validation techniques.
3. The estimation of the expected out-of-sample error.
4. A justification for all modeling decisions made.
5. The use of the final model to predict 20 distinct test cases.

Data exploration and preparation

The following preparation of the data are done. 0. Load data

```
library(caret)

## Loading required package: ggplot2
## Loading required package: lattice
library(dplyr)

##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(ggplot2)
library(lattice)

training_orig <- read.csv("pml-training.csv")
predicting_orig <- read.csv("pml-testing.csv")
```

1. Remove the features, which the percentage of their values “NA” exceed threshold.

```
is_majority_na <- function(col, threshold = 0.7) {
  nr_prop = sum(is.na(col)) / length(col)
  if (nr_prop > threshold) {
```

```

    return(TRUE)
  } else {
    return(FALSE)
  }
}

get_cols_to_remove <- function(dataframe, threshold = 0.7) {
  result_bool <- sapply(dataframe, is_majority_na, threshold = threshold)
  cols <- names(result_bool[result_bool == TRUE])
  return(cols)
}

cols_to_remove <- get_cols_to_remove(training_orig, threshold = 0.7)

training1 <- training_orig %>%
  select(-all_of(cols_to_remove))

predicting1 <- predicting_orig %>%
  select(-all_of(cols_to_remove))

```

2. Remove the features, which the procentage of their values "" exceed threshold.

```

get_cols_with_high_empty_string_prop <- function(dataframe, threshold = 0.7) {
  empty_prop <- colMeans(dataframe == "")
  result_bool <- empty_prop > threshold
  cols_to_remove <- names(result_bool[result_bool == TRUE])
  return(cols_to_remove)
}

cols_to_remove <- get_cols_with_high_empty_string_prop(training1, threshold = 0.7)

training2 <- training1 %>%
  select(-all_of(cols_to_remove))

predicting2 <- predicting1 %>%
  select(-all_of(cols_to_remove))

```

3. manuell remove features

```

manual_to_remove <- c("user_name", "raw_timestamp_part_1", "raw_timestamp_part_2", "cvtd_timestamp", "X")
training3 <- training2 %>%
  select(-all_of(manual_to_remove))
predicting3 <- predicting2 %>%
  select(-all_of(manual_to_remove))

```

4. For some missing values of the filtered features, I will assign the average value from the same feature

```

impute_with_mean <- function(col) {
  avg_value <- mean(col, na.rm = TRUE)

  col[is.na(col)] <- avg_value

  return(col)
}

training <- training3 %>%
  mutate_if(is.numeric, impute_with_mean)

```

```

predicting <- predicting3 %>%
  mutate_if(is.numeric, impute_with_mean)

training$classe <- as.factor(training$classe)
predicting$problem_id <- as.factor(predicting$problem_id)

```

3. Divide the training set to training and testing set

```

inTrain = createDataPartition(training$classe, p=0.7, , list=FALSE)
training = training[ inTrain,]
testing = training[-inTrain,]

```

Model building

I choose decision tree based approach because of the following reasons:

1. Classification Task: The target variable (classe) is categorical (A, B, C, D, E). The decision tree method is a classification algorithm designed to predict these categories accurately. Multivariate regression is used for continuous, numerical outcomes.
2. High Dimensionality: The dataset has hundreds of features. The decision tree method, such as Random Forest, handles high-dimensional data robustly, naturally dealing with feature selection and overfitting.
3. Performance: In practice, Random Forest consistently achieves higher accuracy (often near 100%) on this specific dataset compared to regression-based approaches.

I prepare cross validation. I use standard k-folder cross validation, where k is 5 and it repeats only once because of restriction of running time.

```

set.seed(1000)
control_params <- trainControl(
  method = "repeatedcv",
  number = 5,
  repeats = 1
)

```

I use the “Recursive Partitioning and Regression Trees” (rpart) method to train the data

```

modFit_rpart <- train(
  classe ~ .,
  data = training,
  method = "rpart",
  trControl = control_params
)
print(modFit_rpart)

```

```

## CART
##
## 13737 samples
##    52 predictor
##    5 classes: 'A', 'B', 'C', 'D', 'E'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 1 times)
## Summary of sample sizes: 10988, 10989, 10990, 10991, 10990
## Resampling results across tuning parameters:
##
##   cp          Accuracy   Kappa

```

```
## 0.03906012 0.5059292 0.35473550
## 0.05899705 0.4150265 0.20766982
## 0.11443393 0.3304226 0.07048968
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.03906012.
```

```
res_rpart <- predict(modFit_rpart, newdata=testing)
conf_matrix <- confusionMatrix(res_rpart, testing$classe)
res <- conf_matrix$overall['Accuracy']
cat("\nAccuracy:\n")
```

```
##
## Accuracy:
cat(res, "\n")
```

```
## 0.4924757
cat("\nOut of sample error:\n")
```

```
##
## Out of sample error:
cat(1 - res, "\n")
```

```
## 0.5075243
```

I use the “Random Forest” (rf) method to train the data

```
modFit_rf <- train(
  classe ~ .,
  data = training,
  method = "rf",
  trControl = control_params
)
print(modFit_rf)
```

```
## Random Forest
##
## 13737 samples
## 52 predictor
## 5 classes: 'A', 'B', 'C', 'D', 'E'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 1 times)
## Summary of sample sizes: 10990, 10989, 10990, 10989, 10990
## Resampling results across tuning parameters:
##
## mtry Accuracy Kappa
## 2 0.9901727 0.9875674
## 27 0.9903910 0.9878439
## 52 0.9834027 0.9790015
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 27.
```

```
res_rf <- predict(modFit_rf, newdata=testing)
conf_matrix <- confusionMatrix(res_rf, testing$classe)
```

```
res <- conf_matrix$overall['Accuracy']
cat("\nAccuracy:\n")
```

```
##
## Accuracy:
cat(res, "\n")
```

```
## 1
cat("\nOut of sample error:\n")
```

```
##
## Out of sample error:
cat(1 - res, "\n")
```

```
## 0
```

I use the “Gradient Boosting Machine” (gbm) method to train the data

```
modFit_gbm <- train(
  classe ~ .,
  data = training,
  method = "gbm",
  trControl = control_params,
  verbose = FALSE
)
print(modFit_gbm)
```

```
## Stochastic Gradient Boosting
```

```
##
## 13737 samples
## 52 predictor
## 5 classes: 'A', 'B', 'C', 'D', 'E'
##
```

```
## No pre-processing
## Resampling: Cross-Validated (5 fold, repeated 1 times)
## Summary of sample sizes: 10988, 10990, 10991, 10990, 10989
## Resampling results across tuning parameters:
```

```
##
## interaction.depth n.trees Accuracy Kappa
## 1 50 0.7479053 0.6802182
## 1 100 0.8204850 0.7726368
## 1 150 0.8544072 0.8155953
## 2 50 0.8509127 0.8110364
## 2 100 0.9056555 0.8805788
## 2 150 0.9288050 0.9099101
## 3 50 0.8942989 0.8661635
## 3 100 0.9403062 0.9244467
## 3 150 0.9604713 0.9499838
##
```

```
## Tuning parameter 'shrinkage' was held constant at a value of 0.1
```

```
##
## Tuning parameter 'n.minobsinnode' was held constant at a value of 10
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were n.trees = 150, interaction.depth =
```

```
## 3, shrinkage = 0.1 and n.minobsinnode = 10.
res_gbm <- predict(modFit_gbm, newdata=testing)
conf_matrix <- confusionMatrix(res_gbm, testing$classe)
res <- conf_matrix$overall['Accuracy']
cat("\nAccuracy:\n")
```

```
##
## Accuracy:
cat(res, "\n")
```

```
## 0.9725728
cat("\nOut of sample error:\n")
```

```
##
## Out of sample error:
cat(1 - res, "\n")
```

```
## 0.02742718
```

According to the comparison of above 3 models, one can see that random forest method deliver the best results, following by the “gbm”. The “rpart” method gives the worst results.

Prediction:

I use the “random forest” method to predict the new test cases in “pml-training.csv”.

```
res <- predict(modFit_rf, newdata=predicting)
print("\nPredicted results:\n")
```

```
## [1] "\nPredicted results:\n"
print(res)
```

```
## [1] B A B A A E D B A A B C B A E E A B B B
## Levels: A B C D E
```